

Parallel Word Counting in a Text Corpus with MPI

Context

Before starting the lab, the dataset must be generated using the provided generator script (https://github.com/augustosalaraz/MPI/tree/main/word_lab).

```
docker run --rm -v "$(pwd)"/:/app augustosalazar/slim-mpi:2  
python /app/generator.py
```

or

```
docker run --rm -v "%cd%:/app" augustosalazar/slim-mpi:2 python  
/app/generator.py
```

A folder named dataset/ then contain:

- a file consulta.txt, with one word per line,
- multiple text files named file_XXXX.txt, which form the corpus.

The goal is to count how many times each word in **consulta.txt** appears in the corpus and report the top 10 most frequent words.

This lab will be developed in three stages:

1. use a provided sequential implementation as a baseline,
2. build a first MPI version using broadcast and static file distribution,
3. build a second MPI version that reduces load imbalance.

```
docker run --rm -v "$(pwd)"/:/app augustosalazar/slim-mpi:2  
python /app/baseline_secuencial.py
```

Learning Objective

Design, implement, and experimentally evaluate parallel solutions for a text-processing problem, analyzing how the number of processes and the workload distribution strategy affect performance.

Part 0. Baseline Reference

A sequential reference implementation will be provided. All groups must use this implementation as the common:

- correctness baseline,
- timing baseline,
- reference for computing speedup and efficiency.

Students must run the sequential version on the provided dataset and record:

- the execution time T_{seq} ,
- the global output produced by the sequential solution.

Part 1. First MPI Implementation

Implement an MPI solution in which:

1. rank 0 reads consulta.txt,
2. rank 0 broadcasts the query words to all processes using broadcast,
3. rank 0 obtains the list of file_*.txt files,
4. the files are distributed statically among the processes,
5. each process counts locally the occurrences of the query words in its assigned files,
6. partial results are gathered in rank 0,
7. rank 0 builds the global result and prints the top 10.

Your solution must report, for each process:

- the number of assigned files,
- the local processing time.

Part 2. Experimental Evaluation

Run the MPI implementation from Part 1 with different numbers of processes:

$p \in \{1, 2, 4, 8\}$

For each configuration:

1. perform at least 3 runs,
2. record the total execution time,
3. record the local time for each process,
4. compute the average time.

Note: run `docker run --rm augustosalazar/slim-mpi:2 nproc` to see how many cores has the container available and include that data into the analysis

Metrics to compute

Speedup

$$S_p = \frac{T_{seq}}{T_p}$$

Efficiency

$$E_p = \frac{S_p}{p}$$

where:

- T_{seq} is the sequential execution time,
- T_p is the average execution time of the MPI implementation using p processes.

Part 3. Load-Balancing Improvement

Implement a second MPI version that reduces the load imbalance observed in Part 1.

Part 4. Second Experimental Evaluation

Repeat the experiment from Part 2 for the second MPI implementation.

Compare both MPI versions in terms of:

- total execution time,
- speedup,
- efficiency,
- load balance.

Required Analysis

Provide brief, evidence-based answers to the following questions:

1. Did the first MPI implementation improve execution time compared to the sequential baseline?
2. Was the observed speedup linear?
3. Is there evidence of load imbalance? How was it observed?
4. Did the second implementation reduce load imbalance?
5. Did the improved distribution strategy produce a real performance improvement?
6. What limitations affected your experiment?

Deliverables (on English)

A repository with

Code

- generator.py (as provided, no changes)
- baseline_secuencial.py (as provided, no changes)
- run_all.sh (as provided, no changes)
- MPI implementation version 1 (mpi1.py),
- MPI implementation version 2 (mpi2.py).

On the Readme.md

1. Title and Team Information
2. Problem Description
3. Environment and Execution Instructions
4. Experimental plan
 - a. Sequential Baseline (what it does and how)
 - b. MPI version 1 (what it does and how)
 - c. The test procedure
5. Experimental plan execution
 - a. Sequential baseline timing
 - b. MPI version 1 timing results
 - c. Load imbalance evidence
 - d. Implementation of MPI Version 2 correcting the imbalance with its timing results
(total execution time, speedup, efficiency, load balance)
6. Analysis
 - a. Did the first MPI implementation improve execution time compared to the sequential baseline?
 - b. Was the observed speedup linear?
 - c. Is there evidence of load imbalance? How was it observed?
 - d. Did the second implementation reduce load imbalance?
 - e. Did the improved distribution strategy produce a real performance improvement?
 - f. What limitations affected your experiment?
7. Conclusions

State whether the parallel implementations improved execution time compared to the sequential baseline, the most important problem observed in the first parallel version, if the second version helped and how ending with a judgment based on evidence

Evaluation

UNIVERSIDAD DEL NORTE DIVISIÓN DE INGENIERÍAS				
SO (6): an ability to develop and conduct appropriate experimentation, analyze and interpret data, and use engineering judgment to draw conclusions				
Rubric SO (6)	Very Deficient	Deficient	Good	Very good
Plan the experimental procedure based on established guidelines, norms, standards and requirements	En la planificación del experimento no considera las normas, guías, estándares, ni los requerimientos establecidos.	In the planning of the experiment, norms, guidelines and standards are inadequately considered and the established requirements are not met.	Most of the norms, guidelines, standards, and established requirements are considered in the planning of the experiment.	All established norms, guidelines, standards and requirements are considered in the planning of the experiment.
Execute the experimental plan	The experimental plan was not properly executed.	The plan was not executed in order. The measurements are unreliable.	The plan was executed in order. Some reliable measurements were made.	The plan was executed in order. Reliable measurements were made.
Analyze and interpret experimental results	Incorrect calculations or misinterpretation and analysis were made.	The calculations made present conceptual errors and their interpretation and analysis is confusing.	The calculations were made correctly. Interpretation and analysis are incomplete.	The calculations were made correctly. The analysis performed is clear and relevant. The interpretation of the data is adequate.
Generate conclusions based on engineering criteria	The conclusions do not relate to the questions or hypotheses raised.	Some conclusions are not related to the questions or hypotheses raised and are not based on engineering criteria.	Most of the conclusions are related to the questions or hypotheses raised, and are based on engineering criteria.	The conclusions are related to the questions or hypotheses raised, and are based on engineering criteria.